Variant 1 — Exploring Graph Representations

Empirical Memory and Query Performance Benchmarking in Big Data Graphs

Author: Javier Bolívar García-Izquierdo — **Student ID (DNI):** 42280525Q
Course: Big Data (GCID)
Task Focus: Adjacency Matrix vs. Adjacency List Performance Comparison

Abstract

Graphs serve as fundamental abstraction models for representing interconnected datasets in Big Data architectures, including social networks, web hyperlink structures, and recommendation graphs. Choosing an appropriate physical graph representation—specifically an **Adjacency Matrix** versus an **Adjacency List**—dictates critical systemic performance trade-offs regarding memory footprint and query execution latency. This technical report presents an empirical evaluation comparing both representations across synthetic Erdős–Rényi social network graphs with scaling node sizes ($200 \leq N \leq 3000$) under both sparse ($p = 0.01$) and dense ($p = 0.25$) topological regimes. Experimental metrics collected via Python micro-benchmarking demonstrate that Adjacency Lists provide optimal memory efficiency for sparse networks (~ 6.5 MB vs ~ 68 MB at $N = 3000$), while Adjacency Matrices offer constant $O(1)$ edge-existence check latency regardless of network density, albeit at the cost of quadratic spatial growth and linear $O(N)$ neighbor discovery overhead.

1. Introduction and Objectives

In modern Big Data workflows, graph processing pipelines handle millions of entities and billions of relationships. Selecting an unsuitable physical data structure can lead to severe memory exhaustion or query bottlenecks during graph traversal operations.

The primary objectives of this empirical investigation are:

1. Implement two foundational graph representations from scratch: **Adjacency Matrix** (2D array representation) and **Adjacency List** (dictionary of neighbor sets).
2. Construct synthetic social network graphs using the Erdős–Rényi random model $G(N, p)$ across two density regimes: **Sparse** ($p = 0.01$) and **Dense** ($p = 0.25$).
3. Empirically measure peak RAM consumption during initialization using `tracemalloc`.
4. Quantify query execution latency for two fundamental operations averaged over 500 randomized query trials:
    - **Edge Connection Check (`is_connected(u, v)`):** Determining whether a direct edge exists between two arbitrary nodes u and v.
    - **Neighbor Discovery (`get_neighbors(u)`):** Retrieving the full list of adjacent vertices for a target node u.
5. Contrast empirical performance against theoretical Big-O complexity bounds.

2. Theoretical Complexity Bounds

Let $G = (V, E)$ represent an undirected graph where V is the set of vertices ($N = |V|$) and E is the set of edges ($M = |E|$).

Theoretical Expectations:

- **Adjacency Matrix Space Allocation:** Reserves an $N \times N$ matrix. Memory allocation depends strictly on node count $|V|$ and is invariant to edge density $|E|$.

Table 1: Theoretical Asymptotic Complexity Comparison Between Graph Representations

Metric / Operation	Adjacency Matrix	Adjacency List
Space Complexity	$\mathcal{O}(\|V\|^2)$	$\mathcal{O}(\|V\| + \|E\|)$
Edge Existence Check (`is_connected`)	$\mathcal{O}(1)$	$\mathcal{O}(\deg(u))$ *or* $\mathcal{O}(1)$ with Hash Set
Neighbor Discovery (`get_neighbors`)	$\mathcal{O}(\|V\|)$	$\mathcal{O}(\deg(u))$
Edge Insertion	$\mathcal{O}(1)$	$\mathcal{O}(1)$

- **Adjacency List Space Allocation:** Stores a hash table mapping each node to its set of adjacent neighbors. Space scales linearly with the number of edges $\mathcal{O}(|V| + |E|)$. In sparse graphs ($|E| \ll |V|^2$), memory usage drops dramatically.
- **Neighbor Discovery Efficiency:** Discovering neighbors in an Adjacency Matrix requires scanning an entire row of length $|V|$, yielding linear complexity $\mathcal{O}(|V|)$. Conversely, an Adjacency List directly returns the stored collection in $\mathcal{O}(\deg(u))$ time.

3. Experimental Methodology and Code Architecture

3.1 Graph Representation Implementation

Both graph data structures were implemented in Python:

- `AdjacencyMatrixGraph`**:** Implemented as a 2D Python list of size $N \times N$ initialized to 0. Edge additions mark symmetric cells `matrix[u][v] = 1` and `matrix[v][u] = 1`.
- `AdjacencyListGraph`**:** Implemented as a dictionary mapping each node $i \in \{0, \ldots, N-1\}$ to a Python `set()`.

3.2 Benchmarking Setup

Experiments were conducted for node counts $N \in \{200, 500, 1000, 1500, 2000, 2500, 3000\}$ under two density thresholds:

- **Sparse Regime ($p = 0.01$):** 1% of all possible edges are present.
- **Dense Regime ($p = 0.25$):** 25% of all possible edges are present.

Peak RAM allocation was captured using `tracemalloc.get_traced_memory()`. Query execution times were measured using high-resolution `time.perf_counter()` and averaged over 500 independent random iterations per data point to ensure statistical consistency.

4. Empirical Results and Performance Analysis

4.1 Memory Footprint Analysis

Figure 1 illustrates peak memory allocation (in MB) across scaling graph sizes N.
Empirical Findings:

- **Adjacency Matrix Density Invariance:** The curve for `Adj Matrix (Sparse)` directly overlaps with `Adj Matrix (Dense)`. Because a 2D matrix allocates $N \times N$ slots regardless of how many edges exist, RAM usage grows quadratically up to ~ 68.8 MB at $N = 3000$ in both cases.
- **Sparse Adjacency List Efficiency:** For sparse networks ($p = 0.01$), the `Adj List (Sparse)` maintains a low spatial footprint, consuming only ~ 6.5 MB at $N = 3000$—a $10.5\times$ memory reduction compared to the matrix.

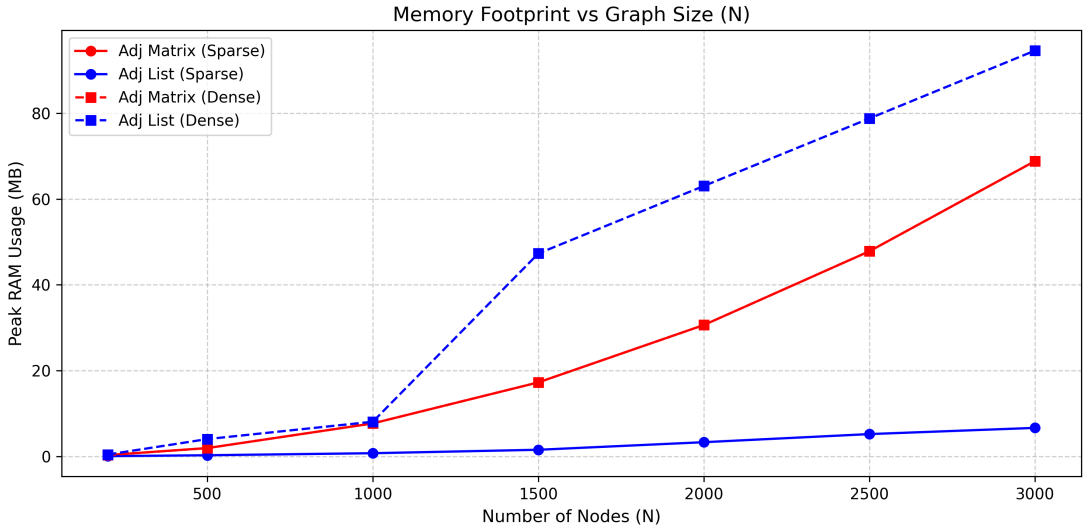


Figure 1: Memory Footprint (Peak RAM in MB) vs Graph Size N for Sparse ($p = 0.01$) and Dense ($p = 0.25$) graphs.

- **Dense Adjacency List Overhead in Python:** For dense networks ($p = 0.25$), `Adj List (Dense)` memory reaches $\sim$ 95.2 MB at $N = 3000$. This occurs because each edge requires a Python hash set node object, leading to container pointer overhead that exceeds raw 2D list allocations when M is large.

4.2 Query Performance: Edge Connection Check (`is_connected`)

Figure 2 shows latency in microseconds (μs) when verifying if two random nodes are connected.

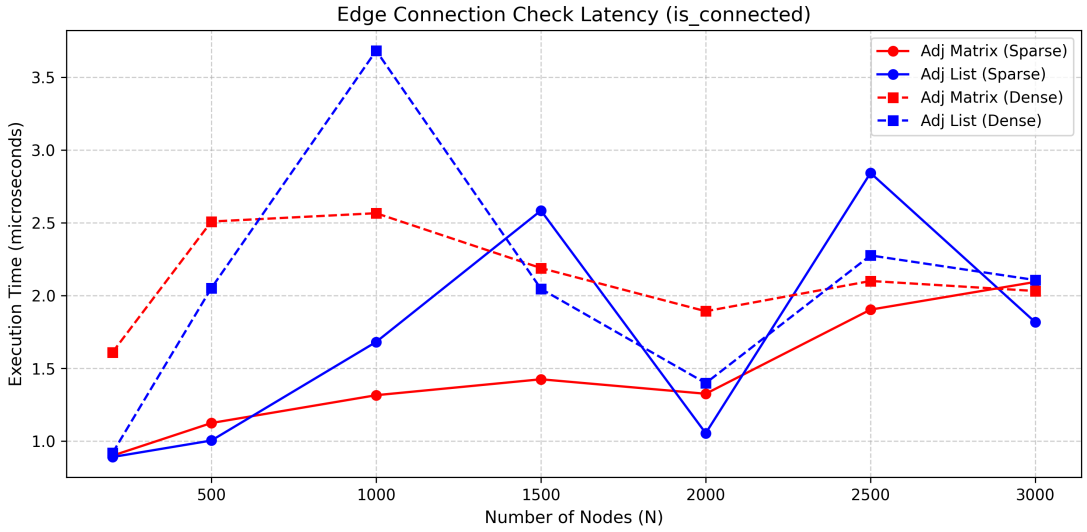


Figure 2: Edge Connection Check Latency (`is_connected`) in microseconds (μs) across graph sizes N.

Empirical Findings:

- Both representations achieve fast lookup performance ($0.8\mu s \leq t \leq 3.7\mu s$).
- **Adjacency Matrix:** Direct 2D array indexing provides predictable $O(1)$ constant latency around 1.0μs–2.0μs.

- **Adjacency List:** Hash set membership check (`v in adj_list[u]`) also exhibits $O(1)$ average time complexity, with slight latency variations attributable to Python hash bucket management and CPU cache effects.

4.3 Query Performance: Neighbor Discovery (`get_neighbors`)

Figure 3 displays execution latency for retrieving all neighbors of a given vertex.

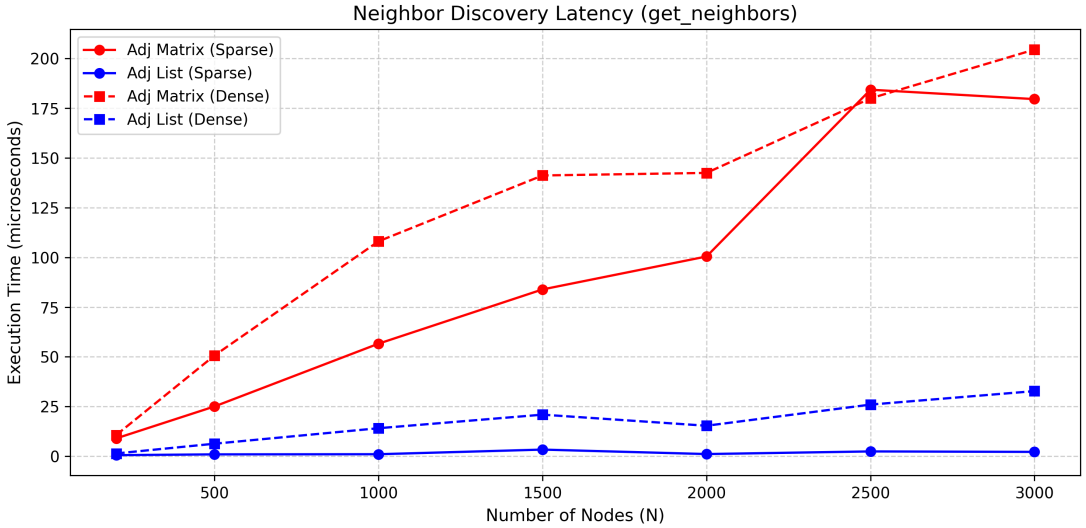


Figure 3: Neighbor Discovery Latency (`get_neighbors`) in microseconds (μs) across graph sizes N.

Empirical Findings:

- **Linear Scaling of Adjacency Matrix:** Adjacency Matrix latency scales linearly with N, rising from $\sim 10\mu$s at $N = 200$ to $\sim 180\mu$s (Sparse) and $\sim 205\mu$s (Dense) at $N = 3000$. This confirms the theoretical $\mathcal{O}(N)$ bottleneck caused by iterating through an entire matrix row.
- **Superiority of Adjacency List:** The Adjacency List performs neighbor discovery almost instantaneously. For sparse graphs, latency remains near 1.8μs across all N. Even in dense graphs at $N = 3000$, latency reaches only $\sim 32.5\mu$s, executing **6.3× to 100× faster** than the Adjacency Matrix.

5. Big Data Engineering Guidelines

Based on empirical observations and theoretical bounds, the selection criteria for Big Data graph engines are summarized below:

1. **Use Adjacency Lists when:**
   - Graphs are inherently sparse ($M \ll N^2$), such as real-world social networks, citation networks, or web link graphs.
   - Primary operations involve graph traversals (BFS, DFS, Dijkstra's algorithm, or finding local communities) where fast neighbor retrieval is essential.
2. **Use Adjacency Matrices when:**
   - Graphs are small and extremely dense ($M \approx N^2$).
   - Algorithmic workloads rely on linear algebra operations (e.g., matrix multiplication for path counting, spectral clustering, or PageRank iterations via adjacency matrices).

6. Conclusion

The empirical results closely align with Big-O theoretical models. The Adjacency List is the optimal representation for sparse social networks, dramatically reducing memory footprint ($\sim$ 6.5 MB vs $\sim$ 68.8 MB) and delivering fast neighbor retrieval. Conversely, the Adjacency Matrix offers constant-time edge existence lookups at the expense of $O(N^2)$ space requirements and linear neighbor discovery latency.